Supplementary Report for the Midterm Project

1 Completed Technical Progress

1.1 Large-Scale F5-TTS Generation Pipeline

A major part of our project has been the construction of a large-scale synthetic speech generation pipeline based on F5-TTS. This was not a simple benchmark reproduction. Instead, we wrote a substantial amount of supporting code from scratch to make large-scale generation feasible for our setting, including the inference scaffolding, batch synthesis pipeline, and analysis scripts for the generated data.

Using this pipeline, we carried out large-scale synthesis on the Emilia dataset, which contains approximately 19,738,862 Chinese samples and 18,099,054 English samples. Our initial large-scale generation run was performed with 16 A100 GPUs over about one week. This stage required significant engineering effort and computational resources, and it formed the basis for our later spoofing experiments and dataset construction.

Because these intermediate engineering steps were not included in the original 3-page midterm report, the actual amount of completed work may not have been fully visible. We therefore include them here to better reflect the technical progress already achieved.

1.2 Engineering Modifications and Debugging

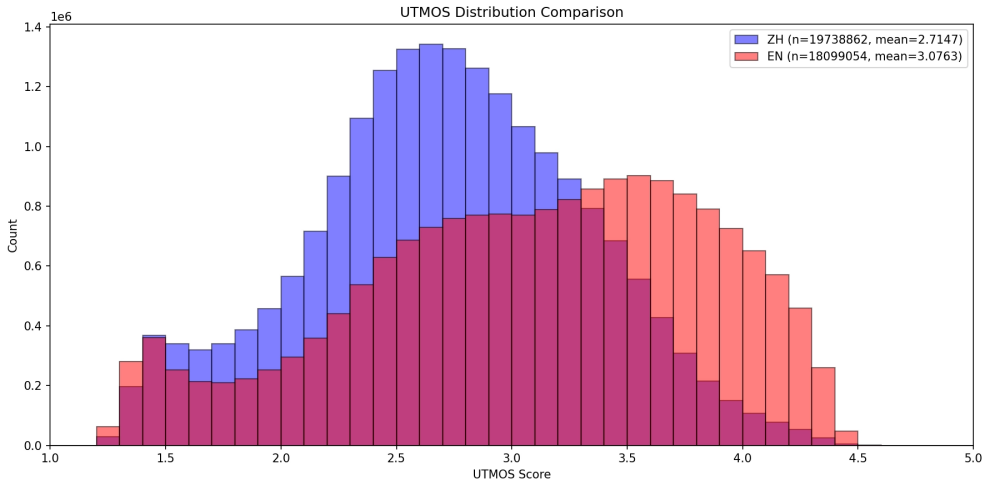
During large-scale generation, we discovered an important issue in the original F5-TTS configuration: the default setting had attention masking disabled,

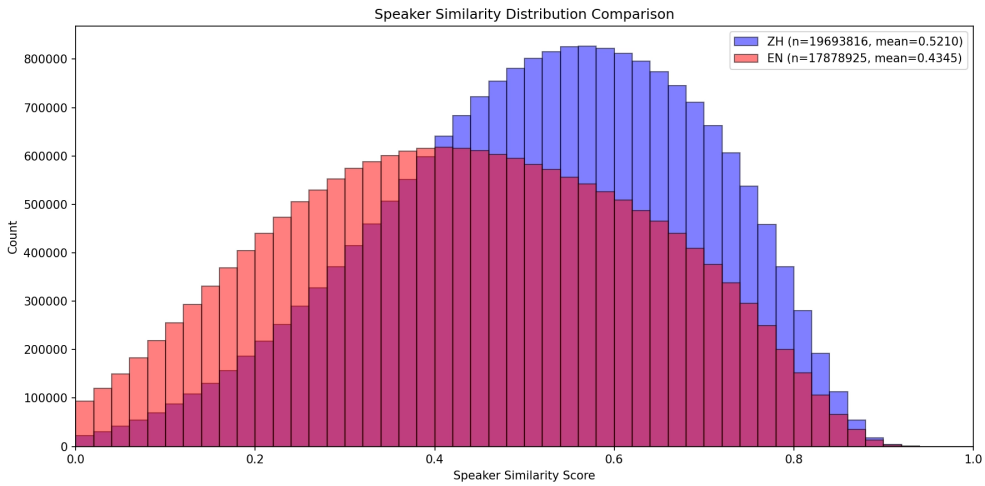
```
arch:
    attn_backend: torch
    attn_mask_enabled: False
```

which led to poor synthesis quality in our first generation run. After identifying this problem, we revised the codebase, enabled attention masking, and further modified the DiT implementation by replacing the original `torch` attention backend with `flashattn`. This reduced GPU memory usage during inference and improved generation efficiency.

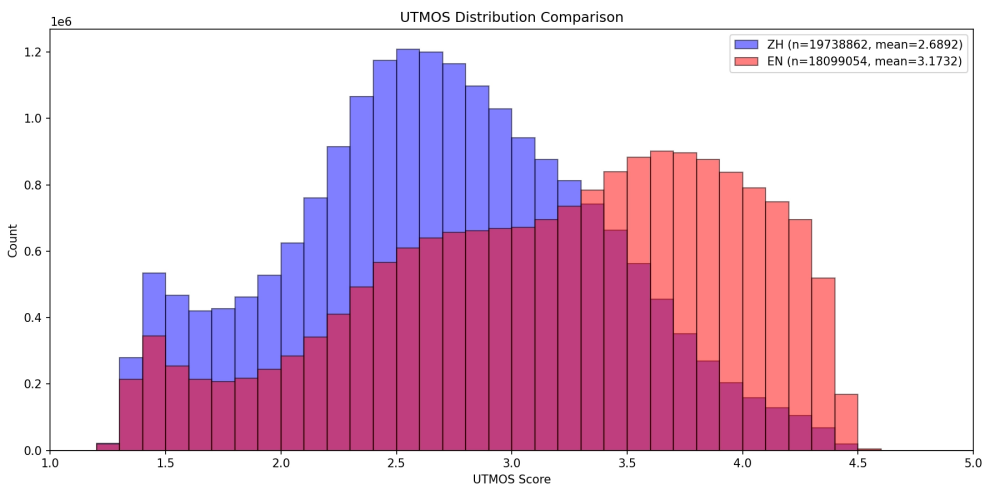
The difference before and after this fix can be seen in the following results.

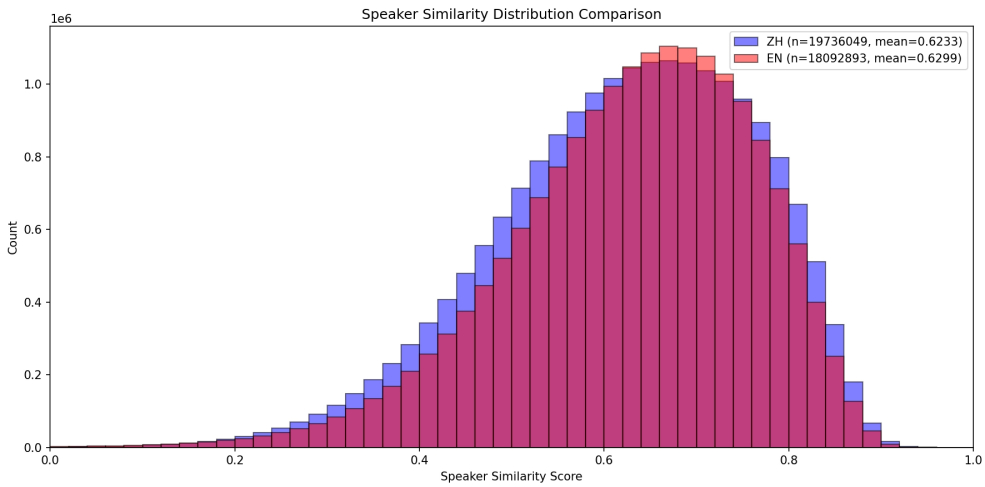
Before fixing the configuration (`attn_mask_enabled=False`, `attn_backend=torch`):





After enabling attention masking and switching to FlashAttention:





The quantitative comparison is shown below.

Table 1: Comparison before and after fixing the F5-TTS attention configuration.

Mean	UTMOS (ZH)	SIM (ZH)	UTMOS (EN)	SIM (EN)
old	2.7147	0.5210	3.0763	0.4345
fixed	2.6892	0.6233	3.1732	0.6299

Although the UTMOS change is relatively small, the speaker similarity scores improve substantially after the fix, especially for English. This strongly supports our conclusion that enabling attention masking and replacing the original attention backend were necessary corrections rather than minor implementation changes.

1.3 SIM/UTMOS-Based Quality Screening

To evaluate and screen the generated data, we also implemented our own batch evaluation pipeline for SIM and UTMOS from scratch. These components were **not provided in the original F5-TTS codebase**. They were written by our team in **February 2026** specifically for this project, and they required substantial engineering time and effort.

In particular, the following files were newly implemented by us and are not part of the original F5-TTS repository:

- **Evaluation**
  - src/f5_tts/eval/eval_metric.py
  - src/f5_tts/eval/evaluator_metric.py
- **Inference**
  - src/f5_tts/model/inferencer_gp.py
  - src/f5_tts/train/inference_gp.py

These modules were essential for our project. The evaluation code enabled large-scale batch quality screening of synthesized speech using SIM and UTMOS, which the original F5-TTS pipeline

did not support. The inference code was also extended by us to support our large-scale generation setting. Together, these additions made it possible for us to run synthesis, evaluate the generated audio, and identify quality issues at scale.

We would like to emphasize that this work was completed by our team in February 2026 and required a significant amount of time and effort. These intermediate implementation details were omitted from the original report mainly because of the strict page limit, which may have created the impression that little work had been done. We provide these details here so that the actual scope of our completed technical work can be more fairly evaluated.

2 Spoofing Experiment Setup Plan

2.1 Real / Non-Targeted Fake / Targeted Fake Data

Our spoofing experiment setup is based on the Emilia dataset and the corresponding synthetic speech generated by our F5-TTS pipeline. Since each sample in Emilia is associated with speaker-level metadata, the data can be naturally organized by speaker for spoofing evaluation.

In this setup, the original Emilia utterances are used as **bona fide (real) speech**, while the generated utterances are used as **spoofed (fake) speech**. We further distinguish between two types of fake data. **Non-targeted fake speech** refers to synthetic speech that is not specifically generated to impersonate the current enrollment speaker. **Targeted fake speech** refers to synthetic speech generated with the explicit goal of imitating the enrollment speaker's identity. This distinction allows us to evaluate both general spoofing effects and more realistic targeted attacks.

2.2 Target-Speaker Selection Criteria

We use the `speaker` field provided in Emilia to group utterances by speaker and select target speakers for spoofing evaluation. To keep the initial setup controlled, we first focus on a single language subset, such as English.

A speaker is selected as a target speaker only if the corresponding data satisfy several basic conditions, including a sufficient number of available utterances, enough total speech duration, reasonable utterance length, and acceptable speech quality. We use metadata such as `duration` and `dnsmos` to perform this preliminary filtering. In the initial stage, we plan to select a controllable subset of qualified speakers, for example 50–100 target speakers, and later expand the setup to a larger scale.

2.3 Trial Construction for ASV and Anti-Spoofing Evaluation

For each selected target speaker, we construct an evaluation set by splitting the real utterances into disjoint subsets. In the initial setup, we use **three real utterances for enrollment** and **two additional real utterances for genuine testing**.

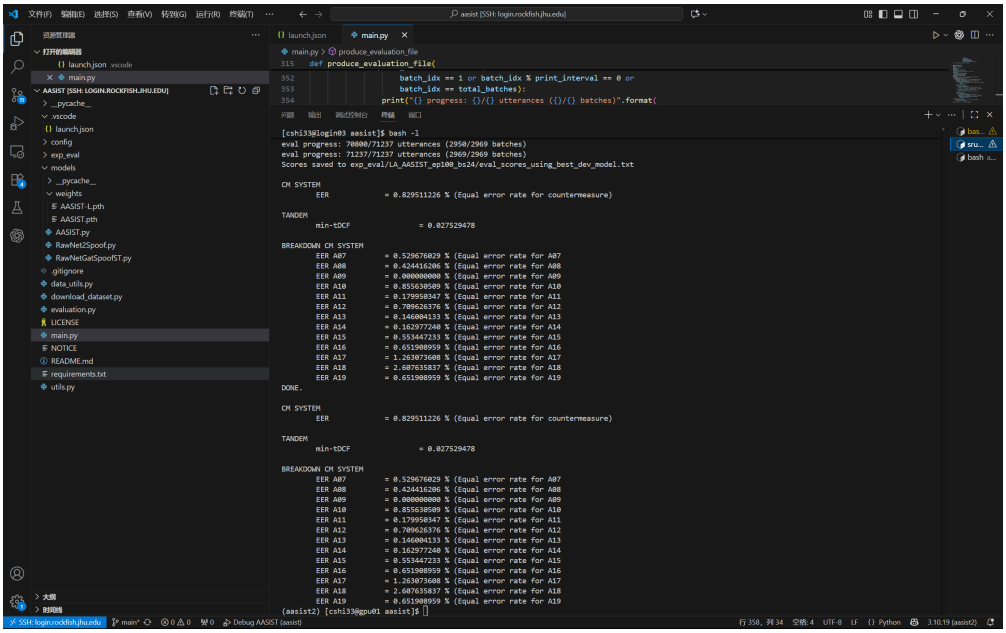
Based on this split, we construct four types of trials. **Genuine trials** pair the enrollment utterances of a target speaker with real test utterances from the same speaker. **Zero-effort impostor trials** pair the enrollment utterances of a target speaker with real utterances from a different speaker. **Non-targeted spoof trials** pair the enrollment utterances of a target speaker with synthetic speech that is not generated to imitate that speaker. **Targeted spoof trials** pair the enrollment utterances of a target speaker with synthetic speech specifically generated to impersonate that speaker.

We evaluate both the **ASV system** and the **AASIST anti-spoofing model** under this protocol. On the ASV side, we analyze whether spoofed speech can be incorrectly accepted as the target speaker. On the anti-spoofing side, we evaluate whether synthetic speech can be correctly detected as fake. Our final goal is to study the joint threat scenario: whether synthetic speech can simultaneously fool the ASV system and evade detection by AASIST.

3 AASIST Reproduction and Preliminary Results

We have already made AASIST work and reproduced its reported performance on ASVspoof 2019 LA. The reproduced result is essentially identical to the original paper: the paper reports **0.83% EER** and **0.0275 min-tDCF**, while our result is **0.8295% EER** and **0.0275 min-tDCF**.

Table 2: AASIST reproduction results on ASVspoof 2019 LA.

	EER (%)	min-tDCF
In paper	0.83	0.0275
Our result	0.8295	0.0275

This near-perfect match shows that our AASIST reproduction is correct and that the anti-spoofing baseline for our project has already been established. We did not include these preliminary results in the original midterm report mainly because of the strict 3-page limit and because we chose to prioritize the large-scale synthetic data construction pipeline, which is more central to our final project.